Penetration Testing Report – Juice Shop

Client: OWASP Juice Shop

Assessment Type: Web Application

Assessment Period: 6/5/2026–6/12/2026

Report Date: 6/12/2026

Assessment Team: Duweling Cyber Consulting

Point of Contact: Email

Executive Summary

Overview

OWASP Juice Shop application. The assessment was performed using a black-box approach to identify security vulnerabilities that could be exploited by an attacker. The assessment identified a total of 18 vulnerabilities across Critical, High, Medium, Low, and Informational severity levels. The most severe findings include SQL Injection enabling authentication bypass and full database extraction, an XXE vulnerability allowing server-side file disclosure, and Broken Access Control permitting privilege escalation to administrator level. Immediate remediation is strongly recommended for all Critical and High severity findings, as these represent direct pathways to system compromise and data breach.

Scope

IP: 10.1.213.40

Domain: *juice.com

Assessment Objectives

- Identify security vulnerabilities.
- Validate the effectiveness of existing security controls.
- Assess the potential impact of identified weaknesses.
- Provide remediation recommendations.
- Provide POC for vulnerabilities.

Risk Summary

Severity	Count
Critical	4
High	4
Medium	1
Low	2
Informational	1

Methodology

The assessment followed industry-standard penetration testing methodologies including:

- Reconnaissance
- Enumeration
- Vulnerability Identification
- Exploitation
- Post-Exploitation
- Privilege Escalation
- Lateral Movement (if applicable)
- Reporting

Tools Utilized

- Nmap
- Burp Suite
- Nessus
- Metasploit
- BloodHound
- Wireshark
- Custom Scripts

Scope and Rules of Engagement

In-Scope Targets

http://juice.com **Web App Pentest**

Testing Constraints

- No denial-of-service testing.
- No social engineering.
- No physical security testing.

Findings

Finding ID: PT-001

Title

Technology Stack Disclosure

Severity

Informational

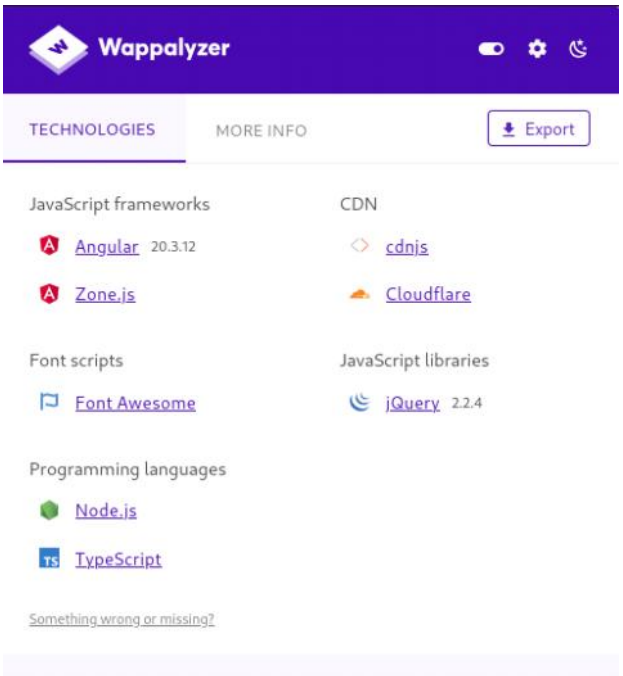
CVSS Score

—----------

Description

The application discloses its underlying technology stack through client-side fingerprinting tools and publicly accessible assets. Observed technologies include Angular, Zone.js, jQuery, Node.js, and third-party libraries served via CDN.

Evidence



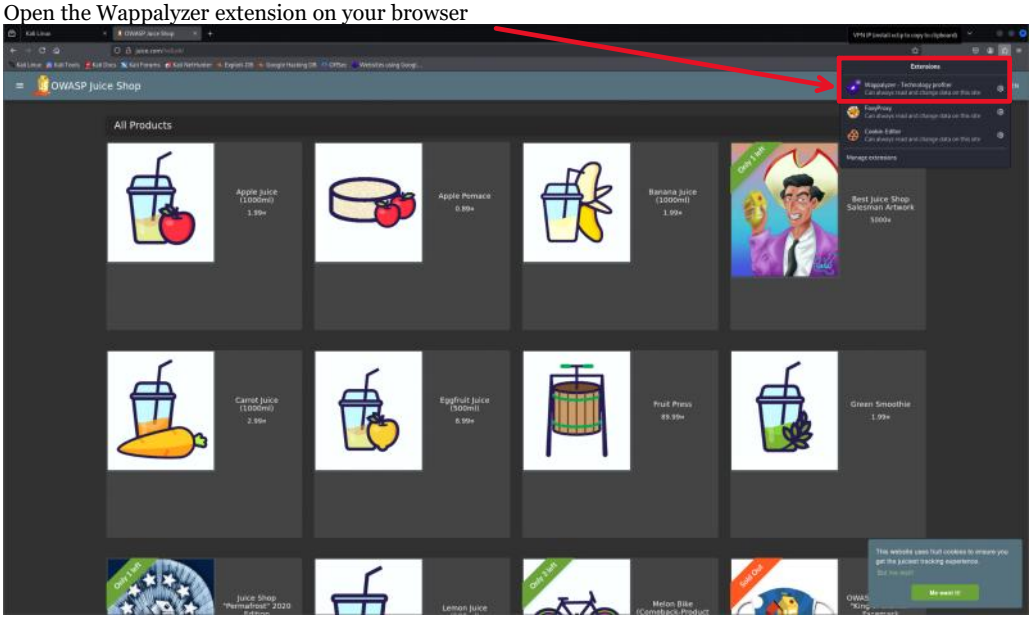
Impact

This information is not directly vulnerable in nature but provides potential attackers with insight into the application's architecture, frameworks, and dependencies. This can assist in targeted reconnaissance and the identification of known vulnerabilities associated with specific framework versions or libraries.

Reproduction Steps

1. Open the Wappalyzer extension on your browser

1. Open the Wappalyzer extension on your browser



Open the browser extension and refresh the page!

Remediation

The application should minimize unnecessary exposure of technology stack information in client-side code and HTTP responses where possible. While complete removal is not feasible in modern web applications, reducing explicit version disclosure and limiting finger printable artifacts can decrease reconnaissance efficiency for potential attackers.

Finding ID: PT-002

Title

Improper Access Control: Directory Listing Enabled on /ftp

Severity

High

CVSS Score

CVSS v3.1: 7.5 (High)

Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Description

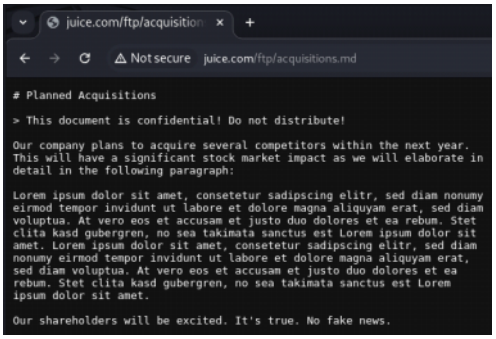
The /ftp directory is publicly accessible without authentication and exposes multiple sensitive files, including backup files, configuration files, and a KeePass password database. This results in a high loss of confidentiality while requiring no privileges, no user interaction, and only low attack complexity.

Affected Assets

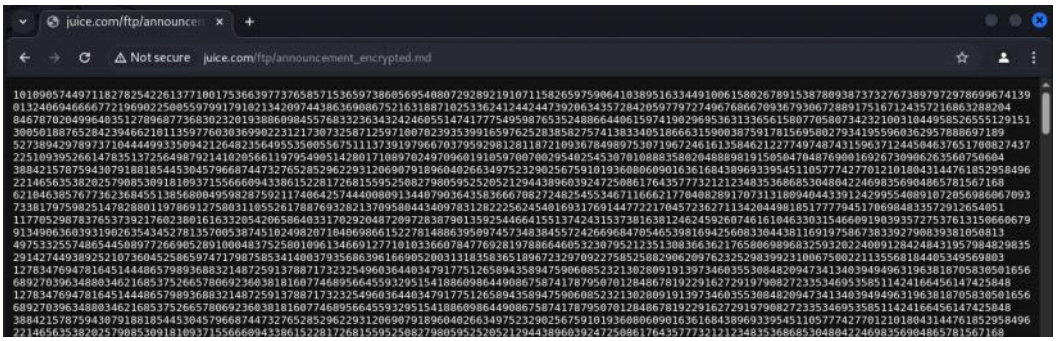
- The web server hosting the /ftp directory
- Sensitive files stored within the directory (backup files, KeePass DB, configuration files, encrypted documents, logs, compiled Python code)
- Any systems whose credentials or secrets may be contained in these files
- Internal application components referenced in exposed configuration or backup files
- Potentially the entire application environment if exposed data enables lateral movement or privilege escalation

Evidence

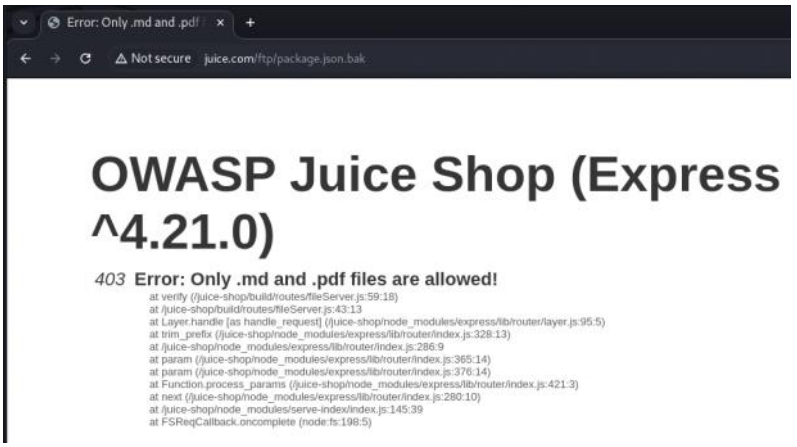
The /ftp/acquisitions.md file contains confidential business information, including planned acquisitions and market-impacting statements. This demonstrates the severity of the information disclosure and the potential for financial, reputational, and regulatory harm.



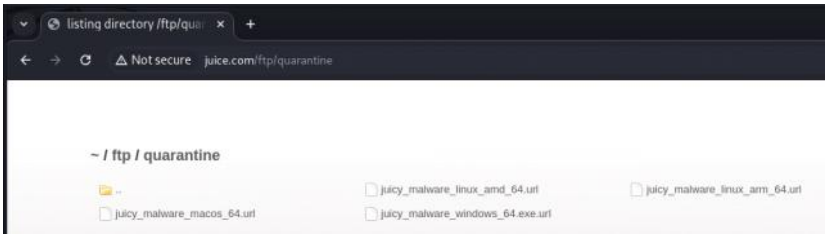
The announcement_encrypted.md file is publicly accessible and contains encrypted internal content. Its presence confirms that sensitive internal documents are exposed due to improper access controls on the /ftp directory.



Attempting to access certain files (e.g., .bak, .json) in the /ftp directory results in a 403 error due to file-type restrictions. This confirms that sensitive files exist and are publicly reachable, even though the application refuses to display them. Their presence reinforces the impact of the directory listing vulnerability.



The screenshot shows the publicly accessible /ftp/quarantine directory, which contains files named to resemble malware samples for multiple operating systems (e.g., juicy_malware_windows_64.exe.url, juicy_malware_linux_amd_64.url). The presence of these files indicates that potentially harmful or quarantined content is stored in a location accessible to unauthenticated users. This further demonstrates the severity of the information disclosure vulnerability and highlights the risk of accidental malware distribution, reputational damage, and potential legal implications.



Impact

The exposure of the /ftp directory results in significant security risks, including disclosure of confidential business documents, leakage of backup and configuration files, exposure of a KeePass password database, and public access to quarantined malware samples. These issues create opportunities for credential compromise, code analysis, insider trading, reputational damage, and potential legal liability. The vulnerability enables attackers to enumerate and download sensitive files without authentication, creating a high likelihood of further system compromise.

Reproduction Steps

1. Perform directory enumeration against the target application using dirsearch:

```
┌──(kali㉿kali)-[~]
└─$ dirsearch -u http://juice.com/ -t 5
```

2. Observe that the tool discovers a publicly accessible /ftp directory:

```
[21:52:55] 200 -   11KB - /ftp
```

This confirms that the /ftp directory is exposed without authentication and can be accessed directly by an unauthenticated user.

Remediation

Disable directory listing and restrict public access to the /ftp directory. Remove all sensitive files from web-accessible locations and store them in secure, access-controlled environments. Implement proper file-type validation, rotate any exposed credentials, and ensure malware samples or quarantined files are not stored in publicly reachable directories. Review server configuration and file permissions to prevent similar exposures in the future.

Finding ID: PT-003

Title

Unauthenticated Access to Prometheus Metrics Endpoint

Severity

Low

CVSS Score

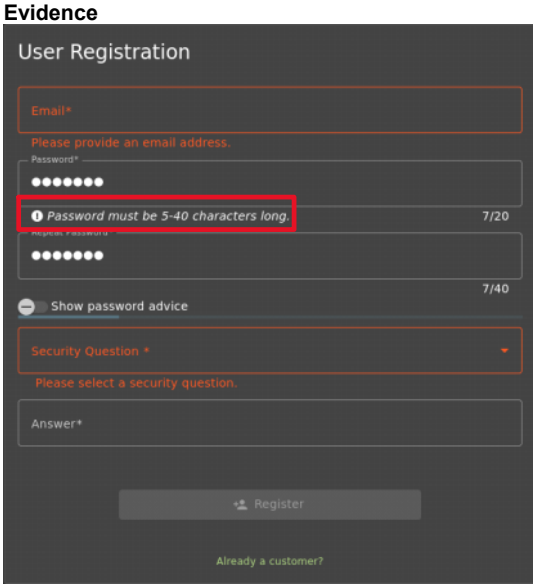
Score: 4.3 (Low)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N

Description

The application's password policy allows users to create passwords that are significantly weaker than industry-standard recommendations. The client-side validation indicates a minimum length of **5 characters** and does not require uppercase letters, digits, or special characters. If these rules are enforced server-side, attackers can more easily guess or brute-force user credentials due to the reduced entropy of allowed passwords. Weak password policies increase the likelihood of successful credential-stuffing attacks and unauthorized account access.

Evidence



Impact

The weak password policy significantly reduces the entropy of user credentials, making them far more susceptible to brute-force and credential-stuffing attacks. Because attackers can attempt authentication over the network without prior access, low-complexity passwords increase the likelihood of unauthorized account takeover. A compromised account may expose sensitive user data, allow manipulation of account-level actions, or enable attackers to pivot deeper into the application depending on the privileges associated with the compromised user. This weakness directly increases the overall attack surface and lowers the cost and effort required for successful exploitation.

Remediation

Implement a stronger server-side password policy that aligns with current industry standards (e.g., NIST SP 800-63B). At minimum, enforce a password length of **8–12 characters** and ensure the backend rejects passwords that do not meet the required complexity criteria. Consider implementing checks against commonly used or previously breached passwords using a vetted password blacklist.
Additionally, ensure rate-limiting and lockout mechanisms are in place to reduce the effectiveness of brute-force and credential-stuffing attacks. All password validation must be enforced **server-side**, regardless of any client-side UI hints or JavaScript checks.

Finding ID: PT-004

Title

Insufficient Password Complexity Requirements

Severity

Low

CVSS Score

Description

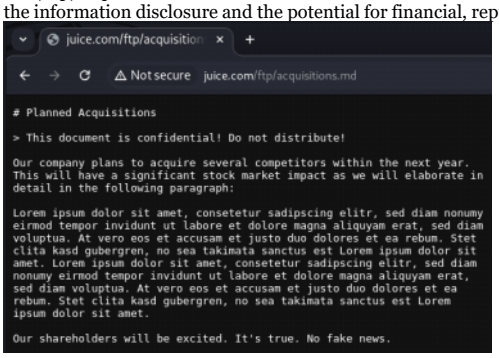
The /ftp directory is publicly accessible without authentication and exposes multiple sensitive files, including backup files, configuration files, and a KeePass password database. This results in a high loss of confidentiality while requiring no privileges, no user interaction, and only low attack complexity.

Affected Assets

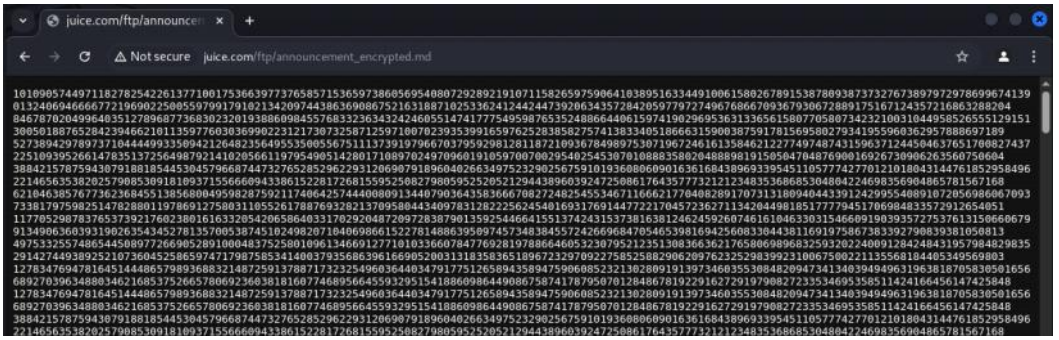
- The web server hosting the /ftp directory
- Sensitive files stored within the directory (backup files, KeePass DB, configuration files, encrypted documents, logs, compiled Python code)
- Any systems whose credentials or secrets may be contained in these files
- Internal application components referenced in exposed configuration or backup files
- Potentially the entire application environment if exposed data enables lateral movement or privilege escalation

Evidence

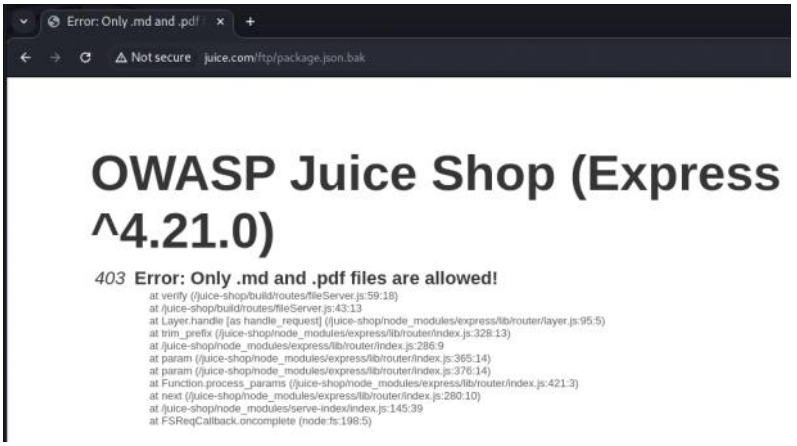
The /ftp/acquisitions.md file contains confidential business information, including planned acquisitions and market-impacting statements. This demonstrates the severity of the information disclosure and the potential for financial, reputational, and regulatory harm.



The announcement_encrypted.md file is publicly accessible and contains encrypted internal content. Its presence confirms that sensitive internal documents are exposed due to improper access controls on the /ftp directory.



Attempting to access certain files (e.g., .bak, .json) in the /ftp directory results in a 403 error due to file-type restrictions. This confirms that sensitive files exist and are publicly reachable, even though the application refuses to display them. Their presence reinforces the impact of the directory listing vulnerability.



The screenshot shows the publicly accessible /ftp/quarantine directory, which contains files named to resemble malware samples for multiple operating systems (e.g., juicy_malware_windows_64.exe.url, juicy_malware_linux_amd_64.url). The presence of these files indicates that potentially harmful or quarantined content is stored in a location accessible to unauthenticated users. This further demonstrates the severity of the information disclosure vulnerability and highlights the risk of accidental malware distribution, reputational damage, and potential legal implications.

Impact

The exposure of the /ftp directory results in significant security risks, including disclosure of confidential business documents, leakage of backup and configuration files, exposure of a KeePass password database, and public access to quarantined malware samples. These issues create opportunities for credential compromise, code analysis, insider trading, reputational damage, and potential legal liability. The vulnerability enables attackers to enumerate and download sensitive files without authentication, creating a high likelihood of further system compromise.

Reproduction Steps

1. Perform directory enumeration against the target application using dirsearch:

```
┌──(kali㉿kali)-[~]
└─$ dirsearch -u http://juice.com/ -t 5
```

2. Observe that the tool discovers a publicly accessible /ftp directory:

```
[21:52:55] 200 -   11KB - /ftp
```

This confirms that the /ftp directory is exposed without authentication and can be accessed directly by an unauthenticated user.

Remediation

Disable directory listing and restrict public access to the /ftp directory. Remove all sensitive files from web-accessible locations and store them in secure, access-controlled environments. Implement proper file-type validation, rotate any exposed credentials, and ensure malware samples or quarantined files are not stored in publicly reachable directories. Review server configuration and file permissions to prevent similar exposures in the future.

Finding ID: PT-005

Title

User Enumeration via Registration Endpoint

Severity

Medium

CVSS Score

Score: 4.3 (Medium–Low)
Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Description

The registration form discloses whether an email address is already associated with an existing account. When a user enters an email that is already registered, the application displays the message **"Email must be unique."** This behavior allows an attacker to differentiate between valid and invalid accounts without authentication. By automating requests to the registration endpoint, an attacker can enumerate a large number of valid user email addresses. This information can be used to facilitate credential-stuffing attacks, password-spraying, targeted phishing, or other account-focused attacks.

Evidence

Impact

The ability to determine whether an email address is already registered allows an attacker to reliably enumerate valid user accounts. Once a list of confirmed accounts is obtained, an attacker can perform targeted authentication attacks such as credential-stuffing, password-spraying, or brute-forcing with significantly higher success rates. This also increases the effectiveness of phishing and social-engineering campaigns, as attackers can focus on known, active users.
If privileged or administrative accounts can be enumerated, the risk increases further, as attackers can directly target high-value accounts for takeover attempts. While enumeration alone does not compromise the system, it meaningfully lowers the cost, effort, and noise required for subsequent attacks.

Reproduction Steps

1. Create a Valid User
2. Attempt to create another user with the same email
3. It doesn't work!

Remediation

Modify the registration workflow so that it does **not** disclose whether an email address is already registered. The application should return a **generic, non-differentiating response** for both existing and non-existing accounts (e.g., "If an account exists, you will receive further instructions").
Validation of email uniqueness should occur **only after form submission**, and any error messages should be phrased in a way that does not reveal account existence.
Additionally, implement server-side rate limiting, IP throttling, and monitoring on registration and authentication endpoints to reduce the feasibility of automated enumeration attacks.

Finding ID: PT-006

Severity

High

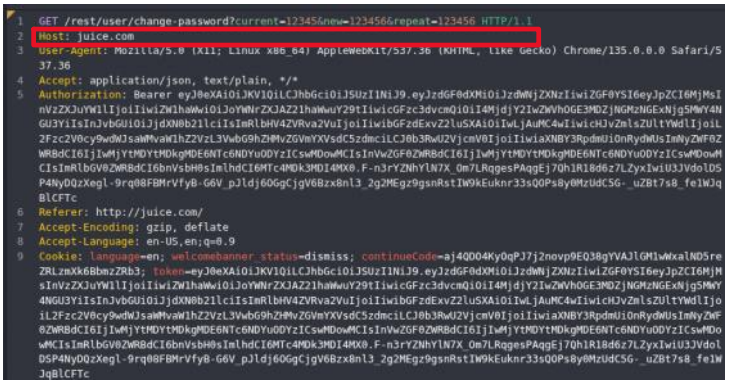
CVSS Score

Score: 7.5 (High)
AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Description

The application implements its password-change functionality using an HTTP **GET** request that includes the **current password**, **new password**, and **password confirmation** as URL query parameters. Because URLs are routinely logged by browsers, servers, proxies, load balancers, CDNs, and monitoring systems, this design results in the exposure of sensitive authentication credentials in multiple locations outside the application's control. Additionally, using GET for a state-changing action violates HTTP semantics and introduces further risks, including CSRF-triggered password changes and unintended caching of sensitive data. The presence of JWT tokens in both the Authorization header and cookies further increases the impact of any log exposure or interception.

Evidence



Impact

The insecure implementation of the password-change functionality results in a **high-risk exposure of authentication credentials**. Because the application transmits the current and new passwords in the URL query string of a GET request, these values are stored in multiple locations outside the application's control, including browser history, server access logs, reverse proxies, load balancers, CDN logs, and monitoring systems. Anyone with access to these logs can obtain valid user credentials and perform full account takeover.
Additionally, the request includes a JWT token in both the Authorization header and cookies, further increasing the severity of any log exposure or interception. The use of GET for a state-changing action also enables CSRF-based forced password changes, allowing an attacker to reset a user's password without their knowledge. Combined, these issues create a scenario where confidentiality is severely compromised and user accounts can be hijacked with minimal effort.

Reproduction Steps

1. Change password on the http://juice.com/#/privacy-security/change-password page, and capture the request with Caido
2. The credentials are revealed in the URL of the get Request

Remediation

The password-change functionality should be redesigned to use a secure HTTP method and avoid exposing sensitive data in URLs. Replace the current GET endpoint with a POST or PUT request that transmits all password fields in the HTTPS request body rather than in query parameters. Ensure that the server rejects any password-change attempts sent via GET. Implement server-side validation for password strength and enforce proper authentication and authorization checks before processing the request. Add CSRF protection (e.g., anti-CSRF tokens or SameSite cookie settings) to prevent unauthorized password-change requests. Finally, ensure that sensitive tokens such as

JWTs are transmitted only in secure, HttpOnly cookies or Authorization headers, and avoid duplicating them across multiple locations.

Finding ID: PT-007

Title

Improper JWT Signature Verification Allows Forged Admin Tokens

Severity

Critical

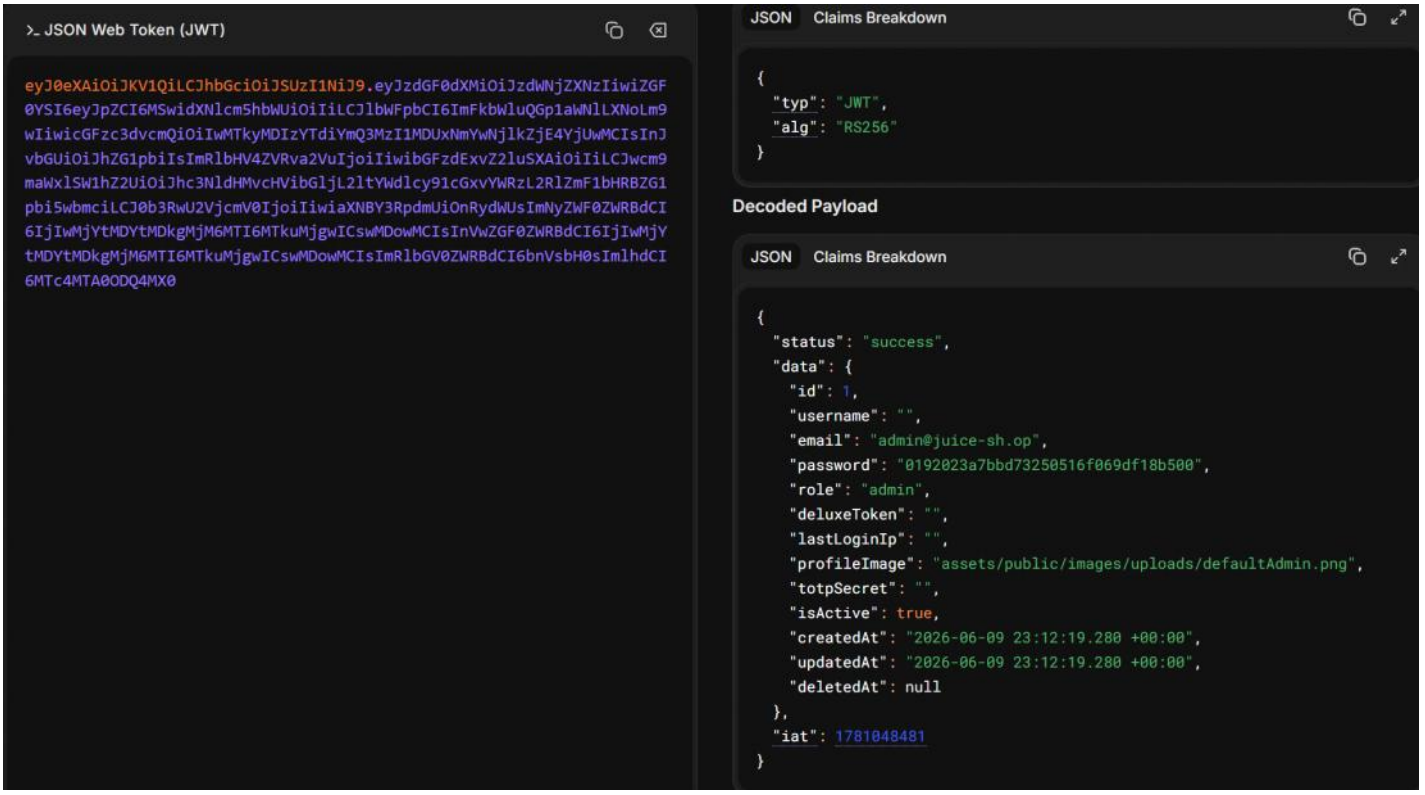
CVSS Score

Base score: 9.8 (Critical)
Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Description

The application improperly verifies JSON Web Token (JWT) signatures for tokens using the RS256 algorithm. Due to a misconfiguration in the verification logic, an attacker can forge arbitrary JWTs— including tokens that impersonate the administrative user—without access to the server's private key. By crafting a malicious token with elevated privileges (for example, role: "admin"), an unauthenticated attacker can gain full administrative access, read and modify sensitive data, and potentially disrupt application functionality. This constitutes a complete compromise of the authentication and authorization mechanism and should be treated as a critical issue.

Evidence



Impact

Exploitation of this vulnerability results in complete compromise of the application's authentication and authorization model. Because the server improperly validates RS256-signed JWTs, an attacker can forge a valid administrative token without possessing the private signing key. This allows an unauthenticated attacker to:

- Authenticate as any user, including the administrative account
- Escalate privileges to full administrator by modifying the role claim
- Access, modify, or delete sensitive data across the application
- Perform administrative actions intended only for trusted users
- Bypass all login, session, and access-control mechanisms
- Maintain persistent unauthorized access by generating unlimited valid tokens

In practical terms, this flaw enables total application takeover. Confidentiality, integrity, and availability are all critically impacted.

Reproduction Steps

- Intercept a valid JSON Web Token (JWT) issued to any authenticated user (e.g., via browser dev tools, proxy interception, or local storage extraction).
- Decode the token using a public JWT decoder such as *jwt.io* to inspect the header and payload.
- Observe that the token is signed using RS256 and that the server incorrectly accepts tokens signed with the public key.
- Modify the token payload (e.g., change "role": "user" to "role": "admin").
- Re-sign the modified token using the application's public key, which should *never* be able to generate a valid signature.
- Submit the forged token to the application and confirm that the server accepts it, granting full administrative access.

Remediation

Implement proper JWT signature verification for RS256 tokens by ensuring the server uses the correct private key for signing and the corresponding public key strictly for verification. The verification logic must reject any token that is not signed with the legitimate private key. Disable support for algorithm downgrades or algorithm substitution (e.g., switching from RS256 to HS256).
Additionally:

- Regenerate a new RSA keypair and revoke any previously exposed or misused keys.
- Enforce strict validation of the alg header to prevent attackers from altering the algorithm.
- Remove sensitive fields (password hashes, TOTP secrets, internal metadata) from JWT payloads entirely.
- Implement short token lifetimes and require refresh tokens or reauthentication.
- Use secure password hashing (bcrypt, Argon2, PBKDF2) instead of MD5.
- Perform server-side authorization checks for all privileged actions, even when a JWT claims admin privileges.

Once fixed, invalidate all existing tokens to ensure forged or improperly validated tokens can no longer be used.

Finding ID: PT-008

Title

Reflected Cross-Site Scripting (XSS) in Search Parameter

Severity

High

CVSS Score

Score: 6.1 (Medium)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

Description

The application is vulnerable to **Reflected Cross-Site Scripting (XSS)** due to insufficient sanitization of user-supplied input in the q search parameter. When a crafted payload is submitted through the search field or appended directly to the URL, the application reflects the input back into the page's HTML without proper encoding. This allows an attacker to inject and execute arbitrary JavaScript in the victim's browser.

Evidence

Impact

The reflected XSS vulnerability allows an attacker to execute arbitrary JavaScript in the browser of any user who clicks a crafted malicious link. Because the application reflects unsanitized input directly into the DOM, an attacker can impersonate the victim, perform actions on their behalf, or steal sensitive data.
Successful exploitation may lead to:

- Session hijacking (stealing session tokens or JWTs)
- Account takeover
- Forced actions (changing user data, making purchases, etc.)
- Phishing or UI redressing within the trusted application context
- Defacement or malicious content injection
- Pivoting into stored XSS or deeper attacks if the attacker can chain the payload

Since no authentication is required and the attack can be triggered via a simple URL, the vulnerability poses a significant risk to all users of the application.

Reproduction Steps

- In the search query type in the POC, `"><img src=x onerror=alert('XSS PoC')>`
- Observe the web application alert box.

Remediation

- Sanitize and encode all user input before reflecting it into the page, using context-appropriate output encoding (HTML, attribute, JS).
- Validate and restrict input for the search parameter to allow only expected characters.
- Use a secure templating engine that automatically escapes variables.
- Implement a Content Security Policy (CSP) to block inline scripts and reduce XSS impact.
- Avoid reflecting raw user input directly into the DOM whenever possible

Finding ID: PT-009

Title

SQL Injection — Authentication Bypass

Severity

Critical

CVSS Score

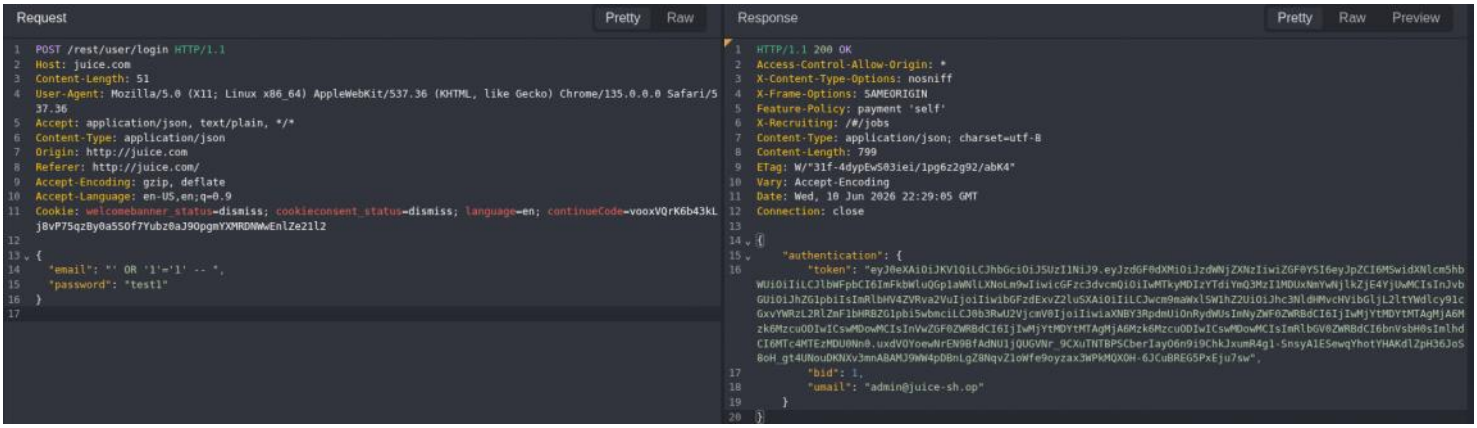
CVSS v3.1: 7.5 (High)

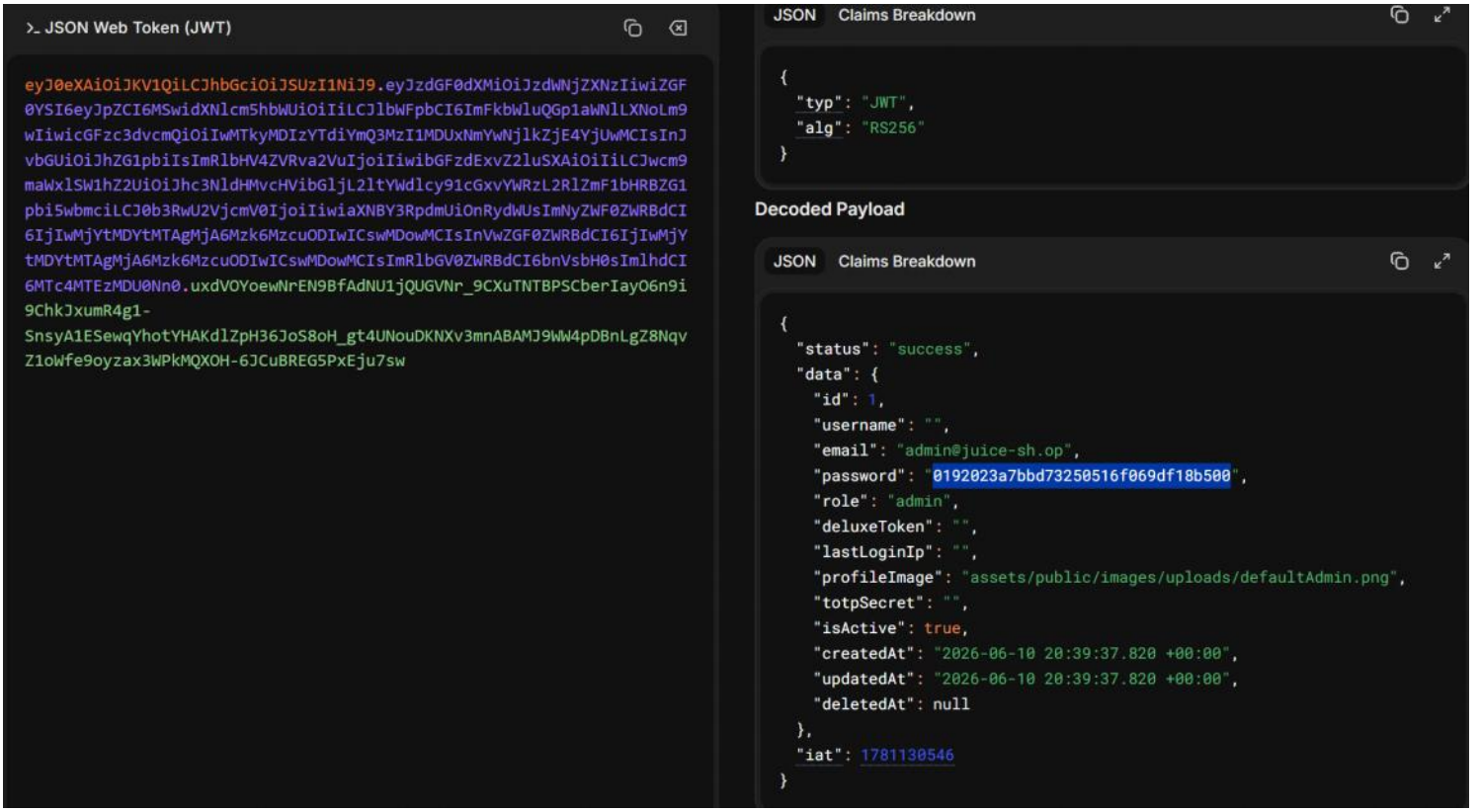
Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

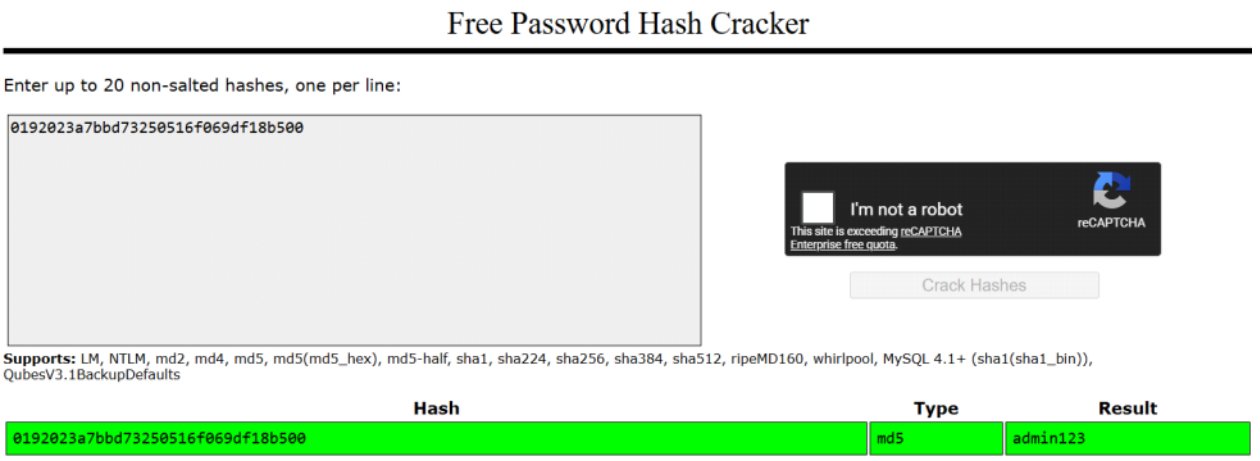
Description

The login endpoint fails to properly sanitize the email parameter, allowing malicious input to be interpreted as part of the underlying SQL query. By supplying a crafted string that alters the query logic, an attacker can bypass authentication and receive a valid JWT associated with an administrative account. The returned token exposes sensitive information, including the admin username and a password hash.

Evidence







Impact

The exposure of the /ftp directory results in significant security risks, including disclosure of confidential business documents, leakage of backup and configuration files, exposure of a KeePass password database, and public access to quarantined malware samples. These issues create opportunities for credential compromise, code analysis, insider trading, reputational damage, and potential legal liability. The vulnerability enables attackers to enumerate and download sensitive files without authentication, creating a high likelihood of further system compromise.

Reproduction Steps

1. Navigate to the login endpoint.

2. In the email field, inject: ' OR '1'='1' —

3. A JWT token is returned containing the admin password hash and username.

4. Decode the JWT and crack the MD5 hash offline to recover plaintext: admin123

Remediation

Use parameterized queries or prepared statements for all database operations so user input is always treated as data, not executable SQL. Adopting an ORM helps enforce this automatically by generating safe queries and reducing direct string-based SQL. Add strict input validation to ensure fields like emails follow expected formats and reject anything abnormal early. Finally, run the application using least-privilege database accounts, limiting what an attacker could do even if input handling fails.

Finding ID: PT-010

Title

Unauthenticated Exposure of Sensitive User Data via /rest/memories Endpoint

Severity

Critical

CVSS Score

CVSS v3.1 Score: 9.8 (Critical)
Vector:

Description

An unauthenticated API endpoint (/rest/memories) exposes sensitive user information without requiring authentication or authorization. The endpoint returns full user objects, including usernames, email addresses, roles, profile image paths, timestamps, and other internal metadata. In some cases, password hashes or tokens may also be exposed. This constitutes a complete breakdown of access control and data protection mechanisms.

Evidence

Impact

The exposure of the /ftp directory results in significant security risks, including disclosure of confidential business documents, leakage of backup and configuration files, exposure of a KeePass password database, and public access to quarantined malware samples. These issues create opportunities for credential compromise, code analysis, insider trading, reputational damage, and potential legal liability. The vulnerability enables attackers to enumerate and download sensitive files without authentication, creating a high likelihood of further system compromise.

Reproduction Steps

1. Navigate to the login endpoint.

2. In the email field, inject: ‘ OR ‘1’=’1' —

3. A JWT token is returned containing the admin password hash and username.

4. Decode the JWT and crack the MD5 hash offline to recover plaintext: admin123

Remediation

Use parameterized queries or prepared statements for all database operations so user input is always treated as data, not executable SQL. Adopting an ORM helps enforce this automatically by generating safe queries and reducing direct string-based SQL. Add strict input validation to ensure fields like emails follow expected formats and reject anything abnormal early. Finally, run the application using least-privilege database accounts, limiting what an attacker could do even if input handling fails.

Finding ID: PT-011

Title

Privilege Escalation via Insecure Role Assignment

Severity

Critical

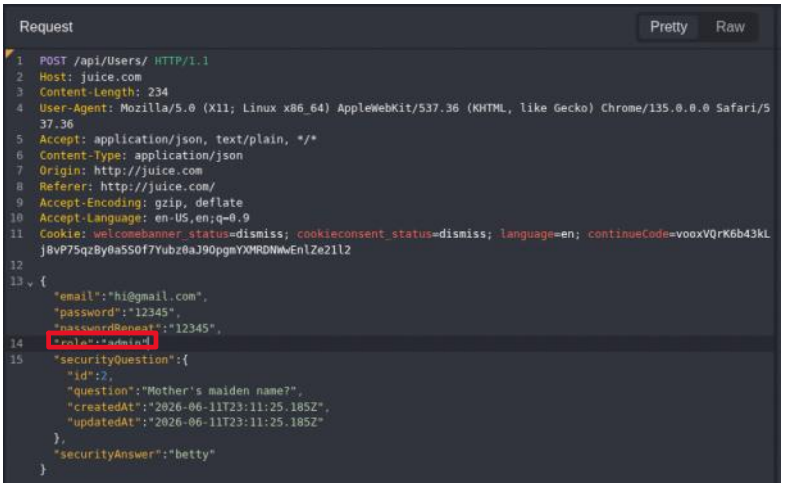
CVSS Score

9.8 (Critical) Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Description

The application’s user-creation endpoint accepts a role parameter directly from client-side input and applies it without any server-side authorization checks. By modifying the registration request and setting "role": "admin", an attacker can create a new account with administrative privileges. This constitutes a vertical privilege escalation flaw caused by insecure role assignment and broken access control. As a result, an unauthenticated attacker can gain full administrative control over the application, including access to sensitive data and system-level functionality.

Evidence



Impact

Exploiting this vulnerability allows an attacker to create an account with administrative privileges without any prior authentication or authorization. With admin-level access, the attacker can fully compromise the application, including viewing, modifying, or deleting sensitive user data, managing other user accounts, altering system configurations, and accessing restricted administrative functionality. This effectively results in a complete loss of confidentiality, integrity, and availability for the affected system. Because the attack can be performed remotely and requires no privileges, the vulnerability poses a critical risk to the entire environment.

Reproduction Steps

1. Register a new regular user with the admin role injected in the HTTP request.

2. Submit and successfully created the new regular user with admin role.

3. Login to verify the account creation.

Remediation

Implement strict server-side authorization controls to prevent clients from assigning privileged roles during account creation. The backend must ignore or reject any user-supplied role field and instead enforce role assignment exclusively on the server based on trusted logic. Only authenticated administrators should be able to create or modify privileged accounts. Additionally, implement an allowlist for acceptable fields during user registration to prevent mass-assignment of sensitive attributes. Review all user-creation and user-update endpoints to ensure that privilege-related fields cannot be manipulated by untrusted input.

Finding ID: PT-012

Title

Insecure Direct Object Reference (IDOR) – Unauthorized Basket Access

Severity

High

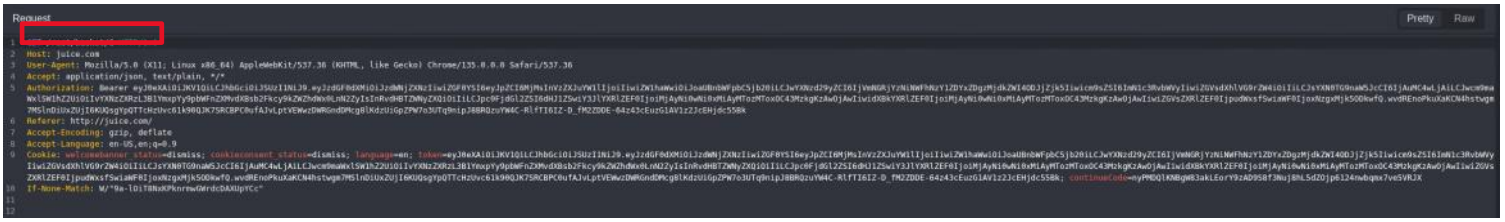
CVSS Score

7.5 (High)
Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

Description

The /rest/basket/{id} endpoint exposes a direct reference to a user's shopping basket through a numeric identifier. The application does not verify whether the authenticated user actually owns the basket associated with the ID provided in the request. By simply modifying the basket ID in the URL (e.g., changing /rest/basket/6 to another number), an attacker can access other users' basket contents. This is a classic Insecure Direct Object Reference (IDOR) vulnerability and represents a failure of server-side access control. As a result, any logged-in user can view data belonging to other users, leading to unauthorized disclosure of personal or transactional information.

Evidence



Impact

Exploiting this vulnerability allows any authenticated user to access the shopping basket of **any other user** simply by modifying the basket ID in the request. This results in unauthorized exposure of another user's personal shopping data, including items they intend to purchase and potentially sensitive product information. If additional endpoints allow item updates or checkout actions, an attacker could also manipulate another user's basket contents or interfere with their purchases. This represents a significant breach of confidentiality and can lead to financial manipulation, loss of user trust, and regulatory concerns related to improper handling of user data.

Reproduction Steps

Steps to Reproduce

1. Authenticate as a regular user.

2. Intercept rest/basket/ID API request and change the ID parameter to another user's ID.

3. Server returns the target user's data without authorization checks.

Remediation

Implement **server-side authorization checks** to ensure users can only access their own basket. Do not trust the basket ID provided by the client. Instead, derive the correct basket from the authenticated user's session or validate ownership before returning any data. Replace predictable numeric IDs with unguessable identifiers (e.g., UUIDs) to reduce enumeration risk.

Remediation Roadmap

Priority	Finding ID	Vulnerability	Severity	Recommended Timeline	Owner
P1	PT-007	Improper JWT Signature Verification	Critical	0–7 Days	Application Development
P1	PT-009	SQL Injection – Authentication Bypass	Critical	0–7 Days	Application Development
P1	PT-010	Unauthenticated Exposure of Sensitive Data (/rest/memories)	Critical	0–7 Days	Application Development
P1	PT-011	Privilege Escalation via Insecure Role Assignment	Critical	0–7 Days	Application Development
P1	PT-002	Directory Listing Enabled on /ftp	High	0–14 Days	Infrastructure / Web Team
P1	PT-006	Password Change via GET Request	High	0–14 Days	Application Development
P1	PT-012	IDOR – Unauthorized Basket Access	High	0–14 Days	Application Development
P1	PT-008	Reflected XSS	High	0–14 Days	Application Development
P2	PT-005	User Enumeration	Medium	30 Days	Application Development
P3	PT-003	Weak Password Policy	Low	60 Days	Identity & Access Management
P3	PT-004	Insufficient Password Complexity	Low	60 Days	Identity & Access Management
P4	PT-001	Technology Stack Disclosure	Informational	90 Days	Application Development